

TwoPuncture 初值求解优化：共享前置阶段

目 录

1 引言：为什么我们要单独看 TwoPuncture	2
2 TwoPuncture 方法在做什么	2
2.1 谱方法求解 Einstein 约束方程	2
2.2 为何它不独立评分却重要	2
3 编译优化：成本最低的第一步	3
3.1 优化等级 -O2 与 -O3	3
3.2 -march 与 -mtune：为你的目标架构说话	3
3.3 自动向量化	3
3.4 OpenMP 编译与运行时选项	3
3.5 危险的 -Ofast 与 -ffast-math	3
4 编译器与工具链：选错会让前功尽弃	4
4.1 几种常见工具链	4
4.2 mpicxx 与 mpifort 是 wrapper 不是编译器	4
4.3 CMake 缓存与多工具链比较	4
5 初值求解的工程优化方向	4
5.1 计算密集循环的 OpenMP 并行化	5
5.2 检查求解器预条件子	5
5.3 替换或调整数学库	5
5.4 删除高频路径上的动态分配	5
5.5 聚合碎片化的 OpenMP 并行区	5
5.6 固定大小数据预分配复用	5
6 收敛速度与数值稳定性的权衡	5
7 TwoPuncture GPU 化 (Bonus)	6
8 CMake 配置注意：CUDA 架构与 AMSS_OPT	6
8.1 AMSS_OPT 只管 C++/Fortran	6
8.2 CMAKE_CUDA_ARCHITECTURES 要按卡改	6
9 本章你将学会	7
10 要点速查	7
11 小结	7

1 引言：为什么我们要单独看 TwoPuncture

在 HPC101 2025 Lab4 里，我们面对的程序是 **TwoPunctureABE**，它从一份双黑洞初值出发，把演化交给 CPU 路径的 **ABE** 或 GPU 路径的 **ABEGPU**。两条路径都要先跑一个共同的前置阶段，那就是 **TwoPuncture**（双刺初值求解）。它本身不独立评分，可是它的时间会原本本计入 task1 与 task2 两个任务的端到端时间。

这个位置很特殊：它既不在 task1 单独打分点的核心循环里，也不在 task2 的 GPU kernel 里，但它卡在两条路径之前。只要你在调试阶段反复重新生成初值，TwoPuncture 的占比就会很显眼。于是，优化它一次，CPU 与 GPU 两个任务同时受益，这是少数能“一鱼两吃”的优化点。

直觉 不妨把 TwoPuncture 想成一座独木桥：桥本身没人评分，但 CPU 和 GPU 两队人马过桥前都得先走它。把桥修短一点，两队总成绩一起变好。

本讲我们就围绕这座桥展开：它做什么、怎么用编译器和工具链加速、有哪些不依赖物理推导的工程优化方向，以及一条进阶的 GPU 化 bonus 路线。

2 TwoPuncture 方法在做什么

我们先回答“它是什么”，再去谈“怎么优化”。这一节你不需要懂广义相对论的物理推导，只需要知道它是个数值求解过程。

2.1 谱方法求解 Einstein 约束方程

TwoPuncture（双刺方法）由 Ansorg 等人提出，是一种 **spectral method**（谱方法）。它把 **Einstein constraint equations**（爱因斯坦约束方程），即在初始时刻必须满足的几何约束，写成两组 puncture（刺）坐标下的展开式，再用谱基函数把偏微分方程离散成代数方程组，最后用迭代求解器求出系数。

直觉上，谱方法用全局光滑函数逼近解，因此收敛阶很高，但代价是每次迭代都要做大量多项式求值与线性代数运算，这正是计算密集点所在。对优化者来说，物理细节可以黑盒，我们关心的是：求解器在反复迭代，每一步算什么，热点在哪。

2.2 为何它不独立评分却重要

TwoPuncture 的输出是一份初始数据文件，ABE 与 ABEGPU 都要读它。在最终评分里，task1 量的是 CPU 路径端到端时间，task2 量的是 GPU 路径端到端时间，两者都包含 TwoPuncture 阶段。这意味着：

1. 它不单独打分，所以你看不到“TwoPuncture 分数”这一栏；
2. 但它直接出现在两个任务的总时间里，砍掉它等于同时给两个任务提速；
3. 在反复改参数重跑初值的开发循环里，它的占比会被放大，体感更明显。

提示：如果你只盯着 task1 的 ABE 循环优化，可能忽略了 TwoPuncture 这块“共享前置”，结果两个任务都吃了一份额外的固定开销。

例 假设端到端总时间为 100 s，其中 TwoPuncture 占 30%，即 30 s，其余 ABE 占 70 s。我们通过编译优化与循环并行把 TwoPuncture 压到 15 s，新的端到端是 $70 + 15 = 85$ s。对 task1 提速 15%。

再看 task2：假设 ABEGPU 部分只要 10 s，原来端到端 $30 + 10 = 40$ s。TwoPuncture 砍半后变成 $15 + 10 = 25$ s，提速 37.5%。

你会发现，GPU 路径越快，TwoPuncture 这块固定开销的相对占比就越大，优化它的边际收益就越高。

3 编译优化：成本最低的第一步

在动源码之前，先榨干编译器能给你的东西。这是改动最小、风险最低、回报最快的一步。

3.1 优化等级 -O2 与 -O3

GNU 工具链里，-O2 是较稳妥的默认，-O3 会启用更激进的循环变换与向量化。对 TwoPuncture 这种以循环为主的代码，-O3 通常有正向收益，但要注意它有时会让二进制变大、指令缓存压力上升。

3.2 -march 与 -mtune：为你的目标架构说话

-march=native 让编译器按当前 CPU 的指令集生成代码，-mtune=native 则告诉它按当前 CPU 的微架构调度指令。Lab4 的目标是 **Kunpeng 920B**，它是一颗 ARM 架构处理器，所以我们要让编译器走 AArch64 路径，启用对应的 SIMD 指令。

注意：-march=native 在交叉编译或容器里不一定可靠，最好显式写出目标架构，避免编译器误判。

3.3 自动向量化

现代编译器能在 -O2 以上自动识别可向量化的循环，把它变成 SIMD 指令。我们能做的是不挡它的路：循环体内避免复杂控制流、避免假依赖、数据布局连续。后面 ABE 讲义会专门讲怎么“帮”编译器向量化，这里先记住它是免费的午餐。

3.4 OpenMP 编译与运行时选项

如果给循环加了 OpenMP 指令，需要用 -fopenmp (GNU) 或对应选项让编译器识别，运行时再靠 OMP_NUM_THREADS、OMP_SCHEDULE 等环境变量控制。注意，仅设环境变量但没编译进 OpenMP 支持，代码不会真的并行。

直觉 你可以把自动向量化想成编译器在循环里“四条腿一起走”，而 OpenMP 是“多个工人同时搬”。前者免费但要你别挡路，后者要你先开开关。

3.5 危险的 -Ofast 与 -ffast-math

-Ofast 在 -O3 之上加了 -ffast-math，它允许编译器重排浮点运算、合并乘加、假设不存在 NaN/Inf。这对纯性能代码没问题，可对 **iterative solver**（迭代求解器）是危险的：浮点运算顺序变了，舍入误差路径就变了，迭代步数、甚至能否收敛都可能改变。

规则：一旦你对数值相关代码动了 `-ffast-math`，必须重新跑端到端正确性检查，不能只看速度快了就交。

4 编译器与工具链：选错会让前功尽弃

Lab4 允许你选择不同工具链，但 C++、Fortran、MPI 三者必须成套，否则会出现 ABI（应用二进制接口）不兼容。

4.1 几种常见工具链

工具链	C++ / Fortran	特点
GNU	g++ / gfortran	最常见，文档多，社区广
LLVM	clang++ / flang-new	新工具链，向量化较强
Intel oneAPI	icpx / ifx	x86 上常强，注意 ARM 支持有限
Arm Compiler	armclang / armflang	ARM 原厂，针对 Kunpeng 调优好
BiSheng	BiSheng clang	华为，针对 Kunpeng 优化

4.2 mpicxx 与 mpifort 是 wrapper 不是编译器

`mpicxx` 与 `mpifort` 是 MPI（消息传递接口）的封装脚本，背后调用真正的编译器。换工具链不是换 `mpicxx`，而是换它背后的 CXX 与 FC。常见做法是先确定 C++/Fortran 编译器，再让 MPI wrapper 指向它们。

4.3 CMake 缓存与多工具链比较

比较不同工具链时，不要在同一个 `BUILD_DIR` 里反复改 `CMAKE_CXX_COMPILER`，CMake 的缓存会让旧值残留，导致链接出诡异错误。为每条工具链单独建一个 `BUILD_DIR`，比如 `build-gnu`、`build-llvm`、`build-bisheng`，互不污染。

例 你想比较 GNU 与 BiSheng。正确做法：

```
mkdir build-gnu && cd build-gnu
cmake -DCMAKE_CXX_COMPILER=g++ \
      -DCMAKE_Fortran_COMPILER=gfortran ..
mkdir ../build-bisheng && cd ../build-bisheng
cmake -DCMAKE_CXX_COMPILER=clang++ \
      -DCMAKE_Fortran_COMPILER=flang-new ..
```

错误做法是在同一个 build 目录里反复 `cmake -DCMAKE_CXX_COMPILER=...`，最后链接报“undefined reference”你还找不到原因。

5 初值求解的工程优化方向

编译选项搞定了，我们再看源码层面。下面这些方向都不需要懂物理，纯工程。

5.1 计算密集循环的 OpenMP 并行化

先用 profiler 找出热点循环（常见的是谱基函数求值、矩阵装配），再判断循环间是否有数据依赖。无依赖的循环加 `#pragma omp parallel for` 即可，注意变量作用域用 `private/firstprivate` 控制好。

5.2 检查求解器预条件子

Preconditioner（预条件子）决定了迭代求解器每步的收敛速度。原代码默认的预条件子不一定是当前 case 下最快的。你可以尝试更适合当前物理参数的策略，比如换对角块近似、换 ILU 的填充层级。

直觉 预条件子像给迭代器一个“地图”，地图越准，走得越少。换地图不改变终点，只改变走的步数。

5.3 替换或调整数学库

标量版的 `sin`、`cos`、`exp` 在热点循环里会变成性能杀手。可以用向量化版本（如 **SLEEF**、**libmvec**），或把能合并的运算合并成 `fma`。注意替换后要复算数值，确认仍在容差内。

5.4 删除高频路径上的动态分配

`malloc/free` 或 Fortran 的 `allocate/deallocate` 在每次迭代都调用，会拖慢代码。把固定大小的缓冲区提到循环外预分配，循环内只复用。

5.5 聚合碎片化的 OpenMP 并行区

如果代码里有一串小并行区，每个都要 `fork-join` 一次，开销可能比省下的还多。把它们合并成一个大的并行区，让同组线程一直在线，是常见优化。

5.6 固定大小数据预分配复用

对那些大小在运行时已知的数组，一次性分配好，传引用进函数复用，避免每次进入函数都重新分配。

这些方向不是互斥的，常常是“先 *profile*，再挑最大的痛点动刀”。一次只改一项，便于判断收益来源。

6 收敛速度与数值稳定性的权衡

换预条件子是一种“动结构”的优化，它带来的不是单步加速，而是步数变化。这里有三种可能：

1. 单步更快，步数不变：纯粹的性能胜利，最容易判断。
2. 单步更慢，但步数大幅减少：总时间更短，但每步的数值行为变了，要复算。
3. 单步更快但步数变多：看似快了，实际更慢，且可能引入振荡。

直觉 衡量标准不是“每步多快”，而是“总迭代时间 + 收敛后是否仍满足正确性”。单步指标会骗人。

这正是 TwoPuncture 优化的微妙之处：你不能只盯一个指标。一个好的做法是建立一份“改动 + 总迭代步数 + 端到端时间 + 正确性是否通过”的小表，每次只改一项，记录对比。

例 默认预条件下，TwoPuncture 跑了 120 步收敛，每步 0.2 s，总时间 24 s。换成块对角预条件子后，每步变成 0.3 s（更贵），但只要 60 步就收敛，总时间 18 s。

表面看每步慢了 50%，实际总时间省了 25%，这就是“步数胜利”。但必须确认收敛后的物理量仍在容差内。

7 TwoPuncture GPU 化 (Bonus)

作为 bonus 方向，可以把 TwoPuncture 的计算热点迁移到 GPU。这不是简单“换个编译选项”，而是一次明确的工程任务，你需要在报告里说清楚四件事：

1. **迁移了什么**：是整个求解器，还是只把谱基函数求值或矩阵装配这类计算密集核搬上去；
2. **host 与 device 的数据组织**：哪些数组常驻 device，哪些每步传输，传输量有多大；
3. **是否改变收敛行为**：GPU 的浮点顺序、reduce 顺序可能与 CPU 不同，迭代步数会不会变；
4. **结果是否仍通过正确性检查**：端到端跑完 ABE 或 ABEGPU，物理量是否在容差内。

提示：GPU 化的收益很容易被 *host-device* 传输吃掉。先量出真正的计算热点，再决定搬哪一块。

8 CMake 配置注意：CUDA 架构与 AMSS_OPT

Lab4 的 CMake 里有几个容易踩坑的点。

8.1 AMSS_OPT 只管 C++/Fortran

AMSS_OPT 这个 CMake 选项只影响 C++ 与 Fortran 的编译选项，不会传给 CUDA。也就是说，你开了 AMSS_OPT，CUDA 代码的优化等级仍然按它自己的逻辑走，要单独配。

8.2 CMAKE_CUDA_ARCHITECTURES 要按卡改

CMake 里 CMAKE_CUDA_ARCHITECTURES 被无条件赋值为 80，对应 A100（计算能力 8.0）。如果你的卡是 V100（计算能力 7.0），必须把它改成 70，否则编译出的 kernel 跑不动或跑错。

坑点：因为是“无条件赋值”，命令行 `-DCMAKE_CUDA_ARCHITECTURES=70` 可能被 CMake 内部的赋值覆盖。最稳妥的做法是直接改 CMakeLists 里的那一行，而不是依赖命令行参数。

例 你在 V100 机器上跑，命令行写：

```
cmake -DCMAKE_CUDA_ARCHITECTURES=70 ..
```

但 CMakeLists 里有这么一行：

```
set(CMAKE_CUDA_ARCHITECTURES 80)
```

如果这行是无条件 set（不是 `if(NOT DEFINED ...)` 形式），你的命令行值会被覆盖，编译出的 kernel 仍是 sm_80，在 V100 上无法运行。正确做法是改 CMakeLists 把它改成


```
70, 或改成 if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES) set(CMAKE_CUDA_ARCHITECTURES
80) endif()。
```

9 本章你将学会

1. 解释 TwoPuncture 在 TwoPunctureABE 中的职责，以及它为何虽不独立评分却影响两个任务；
2. 应用 -O2/-O3、-march/-mtune、自动向量化、OpenMP 编译与运行时选项等编译优化；
3. 选择合适的 C++/Fortran/MPI 成套工具链，并用独立 BUILD_DIR 比较；
4. 设计初值求解的工程优化方向，从循环并行、预条件子、数学库、缓冲区复用到并行区聚合；
5. 权衡收敛速度与数值稳定性，判断“步数胜利”是否成立；
6. 尝试 TwoPuncture GPU 化 bonus，并说明迁移内容、数据组织、收敛行为与正确性。

10 要点速查

主题	要点	注意
TwoPuncture 定位	共享前置, 计入两个任务	不独立评分但影响总时间
编译优化	-O2/-O3, -march/-mtune, 自动向量化	Kunpeng 是 ARM, 用 AArch64
-Ofast/-ffast-math	可能改变浮点顺序	必须重新验证收敛与正确性
工具链	GNU/LLVM/Intel/Arm/BiSheng	C++/Fortran/MPI 成套, 避免 ABI 不兼容
mpicxx/mpifort	是 wrapper 不是编译器	换工具链换背后编译器
CMake 多工具链比较	每条工具链独立 BUILD_DIR	同目录改会缓存污染
预条件子	换策略可能改变迭代步数	单步更慢但步数少可能总胜
GPU 化 bonus	说明迁移内容/数据/收敛/正确性	收益易被传输吃掉
AMSS_OPT	只管 C++/Fortran	CUDA 需单独配
CMAKE_CUDA_ARCHITECTURES	A100=80, V100=70	无条件赋值会覆盖命令行

11 小结

TwoPuncture 是 CPU 与 GPU 两条路径的共同前置，优化它一次能同时惠及 task1 与 task2。我们从最便宜的编译优化入手，讨论了 -O2/-O3、-march/-mtune、自动向量化与 OpenMP 选项，也提醒了 -ffast-math 对迭代求解器的风险。在工具链层面，C++/Fortran/MPI 必须成套，比较时用独立 BUILD_DIR 避免缓存污染。源码层面，循环并行、预条件子调整、数学库替换、缓冲区复用、并行区聚合都是不依赖物理推导的工程方向。换预条件子时尤其要注意“步数胜利”是否成立，GPU 化 bonus 则要把迁移内容与正确性说清楚。最后别忘了 CMake 里 CMAKE_CUDA_ARCHITECTURES 在 V100 上要改成 70，且无条件赋值会覆盖命令行。下一讲我们进入 ABE CPU 演化，看并行结构与向量化怎么动。

